SE LAB 7: FUZZ TESTING FOR INPUT VALIDATION BUGS

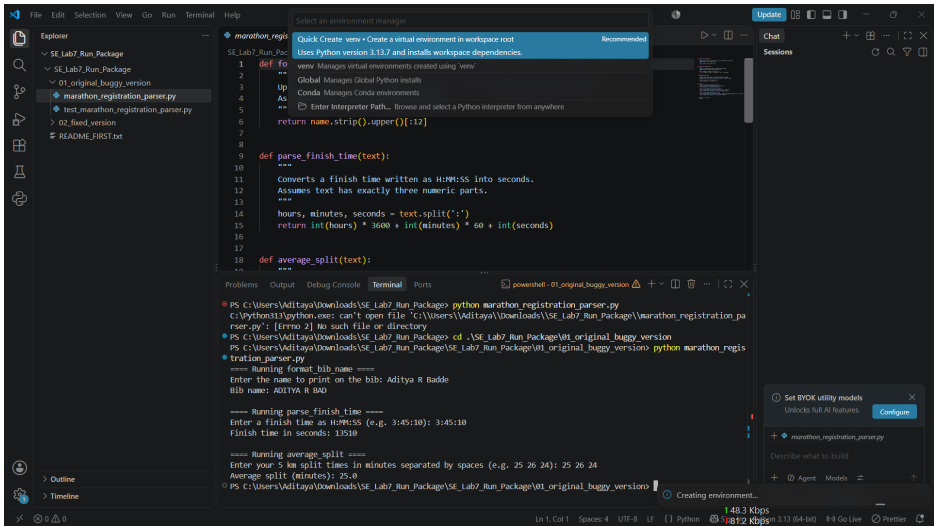
Student: Aditya R Badde | SRN: PES2UG25CS802 | Section: A

Objective	Use Hypothesis-based fuzz testing to discover unexpected-input failures in a Python marathon registration parser, then improve the functions so invalid input is handled safely.
Tools	Python 3.13, Hypothesis, pytest, and Visual Studio Code.

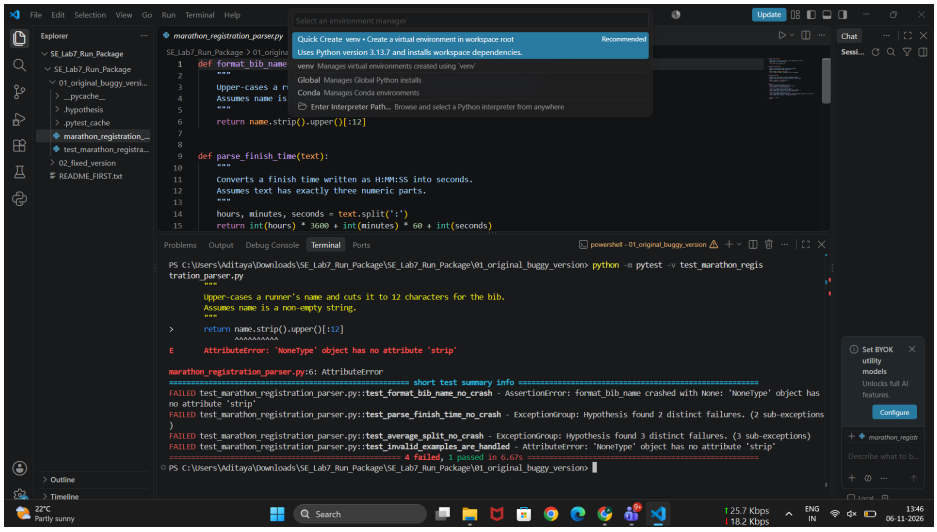
1. Original application and fuzz-test findings

The starter parser assumes inputs are well formed. The original application was run with a runner name, a finish time, and split times. The completed Hypothesis tests then generated unexpected strings and null values. The test run reported **4 failed and 1 passed**, revealing exceptions in the original implementation.

SS1. Original marathon registration application executed with normal sample inputs.



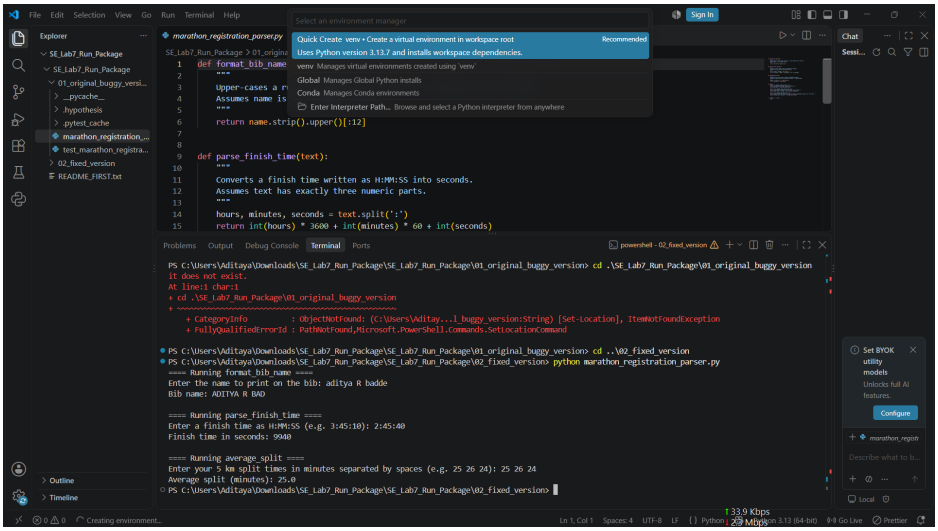
SS2. Hypothesis tests against the original code. Failures include an AttributeError for None and malformed finish-time/split inputs.



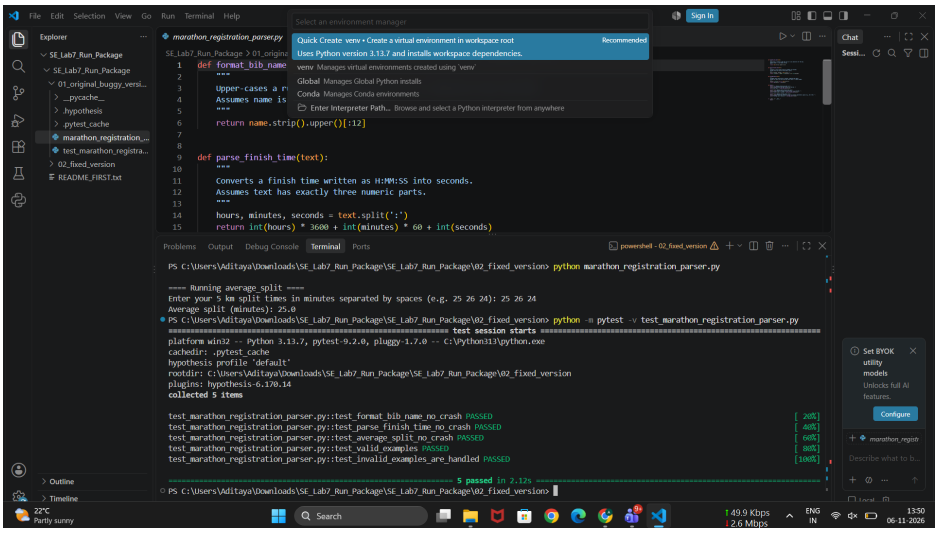
2. Fixed application and verification

The parser was hardened with input-type checks, format validation, numeric conversion handling, and range checks. Invalid bib-name types return an empty string; invalid finish-time and split inputs return None. Valid inputs continue to produce the expected results.

SS3. Fixed application run. The displayed sample outputs include the formatted bib name, finish time in seconds, and average split.



SS4. Final pytest run on the fixed version: all 5 tests passed.



3. Bugs identified and corrections

Function	Issue exposed	Correction
format_bib_name	None or non-string input caused an AttributeError.	Check input type before calling string methods.
parse_finish_time	Malformed values, missing fields, or non-numeric fields caused exceptions.	Check type, require three numeric parts, and validate minute/second ranges.
average_split	Empty or malformed split data caused conversion errors or division by zero.	Reject empty input, catch conversion errors, and reject negative values.

4. Reflection and conclusion

What fuzz testing demonstrated: Testing only typical values can miss failures caused by null, empty, malformed, or unexpected inputs. Hypothesis generated edge cases that were not explicitly written as individual test examples.

Why defensive validation matters: Checking types, structure, numeric content, and valid ranges prevents unhandled exceptions and makes the parser's behaviour predictable when it receives invalid data.

Conclusion: The original test run exposed input-validation defects. After the functions were hardened, the final test run completed with **5 passed**. The supplied four-page lab handout does not include the wording of separate reflection questions, so this report records the learning reflections relevant to the stated objective rather than inventing official questions.